Lecture 13 · Part III · Lecture notes

Transfer learning

Dataset: Flowers102

How to use these notes

The primary text for this lecture is Géron, Chapter 12. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

1 The condition, named

Model	Test accuracy
Always the commonest species	3.87%
Convolutional network from scratch, 4,807,494 parameters	15.3%
What the operator needs	90%

Lecture 12 built that network and it is not close — and its architecture is not the problem, since the same shape of network solves Fashion-MNIST comfortably.

The diagnosis, in one sentence

We asked 1,020 labelled images to pay for 4,807,494 parameters, most of which encode facts about *photographs in general* — edges, textures, colour gradients — rather than facts about flowers.
The repair is therefore not a better architecture, a longer run or a cleverer schedule. It is starting from weights that already work.

Start with what is *not* wrong. The training loop is the same five lines that worked before. The metric is accuracy, anchored against two baselines. The split came with the dataset and the test set was touched once. The architecture is the textbook's own shape. Nothing here is a bug — which is what makes this a different kind of repair from Lecture 2's.

The measurement that names the problem is the gap: 71.8% on the training set against 13.7% on the validation set, 58 points apart. Lecture 5 gave that shape a name — a persistent gap between two plateaus is *variance*, not bias. The network fits the training set and is still climbing; there is nothing wrong with its capacity.

Lecture 5 gave two cures for variance. More training data: no, labelling is the whole cost. More constraint on the hypothesis space: yes, and we have done it once already — weight sharing was a constraint and it bought a factor of 5,478,275. The question is where the next constraint comes from.

1.1 What a pretrained backbone has actually learned

Not "flowers". A hierarchy, and only the bottom of it is general.

Depth	What the filters respond to	Transfers?
first block	edges, colour opponency, simple gradients	to essentially anything
middle blocks	textures, repeated motifs, part-like shapes	to most natural images
last blocks	object-specific configurations	only to similar objects
the head	the 1,000 ImageNet classes	not at all — it is replaced

Which is the whole recipe: keep the bottom, adapt the top, replace the head.

Freezing is not a compromise forced by compute

It is a statement that those layers already encode what you would otherwise be trying to learn from 1,020 images.

Lecture 12 ended with a first layer whose filters were still noise after 364 seconds. The same layer from a network trained on ImageNet shows oriented light–dark boundaries at every angle and opponent-colour blobs — 9,408 numbers, and not one of them is about flowers, or about any of the 1,000 classes that network was fitted on. They are about *photographs*.

ImageNet is 1.28 million labelled photographs over 1,000 classes, none of them our species. Somebody already spent that on learning edges, colours, textures and parts, and those weights are a download. Our 1,020 labels should be spent on the only thing specific to us: which combination of parts is which species.

2 Freeze, unfreeze, and augment

2.1 Probe first, fine-tune second

Two decisions, in this order. A **linear probe** freezes everything, extracts features once and fits a linear classifier — minutes, and it tells you whether the backbone understands your images at all. Then **fine-tune**: unfreeze the last block or two at a small learning rate, and only if the probe was promising.

Why the order is not arbitrary

The probe is a measurement of the *features*, uncontaminated by anything you do to them. If it barely beats the trivial baseline, the features are wrong for this domain and fine-tuning is polishing the wrong object. If it is already close, fine-tuning has little left to buy and may cost you generalisation.

How much data before fine-tuning pays? Fewer than about 20 labelled images per class: probe only, since fine-tuning overfits. Tens: probe, then unfreeze the last block at a small rate. Hundreds: unfreeze progressively with differential rates. Thousands and up: fine-tune everything, and consider training from scratch. Flowers102 gives ten per class, which is the first row — and is why the frozen probe does so much of the work today. These are orders of magnitude rather than thresholds, and the measurement that settles it for your corpus is the probe against the fine-tune on the same validation split.

2.2 The backbone, and its preprocessing

ResNet-18: 11,689,512 parameters, trained on ImageNet, producing 512 numbers before its classifier.

The weights come with their own preprocessing

`ResNet18_Weights.DEFAULT.transforms()` specifies a resize to 256, a centre crop to 224, and per-channel normalisation with mean $[0.485, 0.456, 0.406]$ and standard deviation

$[0.229, 0.224, 0.225]$.
Use *these* statistics, not the ones you computed in the previous lecture. The network's first layer expects inputs on the scale it was trained with, and substituting your own is a silent, uncrashing degradation.

Freezing is two lines — set `requires_grad = False` on every parameter, then replace `net.fc`, because a newly constructed module has `requires_grad=True` and so the head is trainable and nothing else is. That leaves 52,326 trainable parameters.

What `requires_grad = False` actually does: it stops the tensor being a leaf that accumulates a gradient, so `backward()` does not fill `p.grad`. The backward pass still travels through those layers if anything downstream needs it — here nothing does. And it is *not* the same as `eval()`.

Freezing weights does not freeze a network

Batch-norm running statistics are *buffers*, updated in the forward pass whenever the module is in training mode. No gradient is involved, so `requires_grad = False` does nothing to them. A "frozen" backbone in `train()` mode quietly replaces ImageNet's statistics with the statistics of 1,020 flower photographs, in batches of 32. The weights are fixed and the network still drifts — towards a corpus of 1,020 images, which is the opposite of what you froze it for. The fix is `eval()` on the frozen part: the same call as Lecture 10's dropout bug, for the same underlying reason.

If nothing before the head is training, the backbone's output for a given image never changes — so run it once and fit the head on cached features. That is not an optimisation; it is a consequence of freezing. Five seconds for the features and eight for sixty epochs of the head.

Model	Test accuracy	Wall clock
From scratch, 80 epochs	15.3%	364 s
Frozen backbone, linear head	81.2%	13 s

27 times faster and 66 points better. Why a linear model on somebody else's features beats your network: the 52,326 trainable parameters are fitted with 1,020 examples, about 51 parameters per image against 4713 before, while the 11,176,512 frozen ones were fitted with 1.28 million examples by somebody else.

The variance problem was not solved

It was *moved* — to a dataset large enough to absorb it.

2.3 Letting the last block move

The early layers hold edges and colours, which are not about flowers; the late layers hold parts and textures, which partly are. Unfreeze `layer4` and give it a learning rate an order of magnitude below the head's. One optimiser, two parameter groups — that is all a differential learning rate is.

Part of resnet18	Parameters
`conv1` through `layer3`, frozen	2,782,784
`layer4` and the new head, trained	8,446,054

"Only the last block" is not a small thing: three quarters of the parameters are in the block we unfroze. That is Lecture 12's memory calculation mirrored in a design decision — parameters concentrate in the deep, low-resolution layers, so freezing the *general* layers freezes very few of them. What we froze is the part that does general work, not the part that is large.

The two rates have to differ because the head starts random and has everything to learn, while the pretrained block is already good and has something to lose. Give the backbone the head's rate and the first few batches — whose gradients come through a randomly initialised head — will damage the representation you downloaded. It does not crash; the accuracy simply comes out lower and you conclude that transfer learning does not work.

Catastrophic forgetting

The pretrained weights being a good starting point is exactly what makes them fragile: a step large enough to be useful on random weights is far too large for weights that are already nearly right.
Which is why the usual recipe warms up the new head first while everything else is frozen.

2.4 Augmentation, and what each transform asserts

We cannot buy labels; we can show the network a different view of the same labelled image every epoch. A random resized crop teaches scale and position tolerance; a horizontal flip says a flower is still that species mirrored. Each is a transformation *the label is known to survive*, and that is the entire criterion — a modelling decision, not a library call.

What augmentation is not

It is not more information: the mutual information between the corpus and the labels is unchanged. It is a *constraint* — it tells the network the answer must be the same across a family of inputs, which puts it in the same box as weight sharing and as ridge. Lecture 5's second cure, arriving for the third time.
And a transformation the label does *not* survive is a labelling error you manufactured.

Transform	What it asserts	True for flowers?
horizontal flip	mirror image is the same class	yes
random resized crop	a part identifies the whole	usually
colour jitter	hue is not diagnostic	dangerous — colour is a flower feature
vertical flip	upside-down is the same class	often, but not for scenes
rotation	orientation carries no information	yes here

Augmentation trades bias for variance: too little and the model memorises 1,020 images, too much and the training distribution stops resembling the test one — and the symptom of too much

is both curves flat and low, which Lecture 5 named as bias. Start with flip and crop, and add one transform at a time, each a hypothesis about an invariance that can be tested like anything else.

3 A failure condition: one transform, two loaders

It is easy to build one pipeline and hand it to both loaders. The training loader wants random crops and flips; the evaluation loader must not have them.

An augmented evaluation set is not a measurement

It makes the score a *random variable* rather than a function of the weights. Score one fixed model — the fine-tuned one — on the clean validation set and it gives 90.88%; on the augmented set, ten times, the mean is 90.02% with a spread of 1.57 points.
The same weights, the same set, 1.57 points apart. Lecture 10's reviewer question arriving again: a deterministic function of fixed weights and fixed data does not wobble.

Consequence	Size, measured
The reported number is pessimistic	0.86 points low
The number is not reproducible	1.57 points of noise
Model selection uses a noisy criterion	a different epoch chosen

The clean validation curve peaks at epoch 14 and the augmented one at epoch 12, so early stopping on the second keeps the wrong checkpoint. The first consequence is harmless; the third is not, and it is the one nobody notices, *because a pessimistic number feels like the safe kind of wrong*.

The check, in two lines

```
a = evaluate(model, val_loader)
b = evaluate(model, val_loader)
assert a == b, f"evaluation is not deterministic: {a} vs {b}"
```

A comment saying "do not augment the validation set" is read once. This runs every time, costs two forward passes, and fails loudly at the moment the mistake is made rather than three lectures later — and it catches all three causes: augmentation, dropout still active, and a shuffled loader whose batches are averaged rather than pooled.

Three more for the silent-failure catalogue, none of which raises: normalisation statistics computed over the whole dataset, worth 0.25 points here and more on a smaller corpus; augmentation applied to the validation set, worth 1.57 points of noise; and a frozen backbone left in `train()` mode, whose statistics drift with no symptom at all.

4 The ablation

Change	Test accuracy	Gain
from scratch	15.3%	—
+ pretrained backbone, frozen	81.2%	65.9 pts
+ `layer4` unfrozen, no augmentation	86.3%	5.1 pts
+ augmentation	88.0%	1.7 pts

Almost all of the gain is in one row, and saying *which* row is what makes the result useful to somebody else.

What these numbers are not

All three transfer rows are single runs. Lecture 12 measured a seed-to-seed spread of 2.34 points on this dataset, so the 5.1 is a difference and the 1.7 is not — and neither has been replicated.

The wall clocks are for one laptop; the ratios transfer between machines much better than the seconds do. And the pretrained weights saw 1.28 million labelled photographs: that labelling cost is real, it is simply not ours.

Model	Test accuracy	Wall clock
Uniform guess	0.98%	—
Commonest species	3.87%	—
Convolutional net, from scratch	15.3%	364 s
Frozen backbone, linear head	81.2%	13 s
Fine-tuned + augmented	88.0%	176 s
Operator's requirement	90%	—

73 points of accuracy and 27 times less wall clock for the probe alone. Be precise about "cheaper", though: the fine-tune is only 2.1 times faster than from scratch, because it runs a much larger network on much larger images. The frozen probe is the one that is dramatically cheaper — and it is also the one you should run first, every time.

4.1 When the recipe does not work

The domains genuinely differ — medical scans, satellite bands, spectrograms and line drawings are not photographs, and the middle blocks stop being useful sooner. The input is not an image at all: reshaping tabular data into a square does not make convolution appropriate, because there is no locality to exploit. You have plenty of labels, in which case training from scratch catches up and passes a frozen backbone — transfer is strongest exactly where labels are scarce. Or the task needs resolution the backbone discarded, which is Lecture 14's whole problem.

How to tell, cheaply: fit a linear head on frozen features and compare with the trivial baseline. If the frozen features barely beat it, the backbone does not understand your images and no amount of fine-tuning will hide it.

One line to keep

Before you train a vision model from scratch, spend 13 seconds measuring what a frozen pretrained backbone and a linear head already give you. On this application that number was 81.2% against 15.3%.
You are entitled to train from scratch. You are not entitled to do it without knowing what you turned down.

And what we still cannot do: say *where* the flower is, because we have one label per photograph and pooling threw the position away; handle two flowers in one frame; or say "I do not know", which was half the operator's original request.

5 The notebook

What to change

Freeze everything including the final block and refit. The accuracy drops by a few points — which tells you how much of the gain came from adapting the last block rather than from the backbone itself, and separates the two rows of the ablation that are easiest to conflate.

6 Exercises

1. A network fits its training set to 71.8% and its validation set to 13.7%. Name the condition, and say which of Lecture 5's two cures is available when labelling is the dominant cost. [4]

2. Explain why a linear head on frozen features can beat a network trained end to end on the same data, in terms of parameters per labelled example. [5]

3. `requires_grad = False` is set on every parameter of a backbone, and something in it changes anyway during training. State what, why, and the fix. [5]

4. Give two augmentations that are safe for photographs of flowers and one that is not, with the claim about the task each one makes. [4]

5. An evaluation pipeline shares its transform with the training loader. Name the three consequences, and say which is the dangerous one and why. [4]

Summary

Nothing was wrong. The loop, the metric, the split and the architecture were all correct, and the network still reached 15.3% — because 1,020 labelled images were being asked to pay for 4,807,494 parameters, most of which encode facts about photographs rather than about flowers. The gap of 58 points between training and validation is Lecture 5's variance, and with labelling as the dominant cost the only available cure is more constraint.

A frozen backbone and a linear head reached 81.2% in 13 seconds — 66 points better and 27

times cheaper — because the trainable parameters fell to about 51 per labelled image while 11,176,512 frozen ones had been fitted on 1.28 million photographs by somebody else. The variance problem was not solved but moved. Unfreezing the last block added 5.1 points and augmentation 1.7, against a seed-to-seed spread of 2.34 that makes only the first of those a difference.

Three failures on the way, none of which raises: preprocessing with your own normalisation instead of the weights' own; a frozen backbone left in `train()` mode, whose batch-norm buffers drift towards 1,020 flowers because they are not parameters and freezing gradients does not touch them; and one transform handed to both loaders, which cost 1.57 points of noise and selected a different epoch.